

Análisis y Diseño de Clases para el Proyecto UdeAStay

Daniel Rúa y Tomás Mesa

Facultad de Ingeniería

Departamento de Ingeniería Electrónica y Telecomunicaciones

Universidad de Antioquia

Medellín, Colombia

Resumen—Este documento presenta el análisis y diseño de la solución orientada a objetos para el proyecto UdeAStay, un sistema de administración para un mercado de estadías hogareñas. Conforme a una directriz de diseño que evita el uso de herencia, se detallan las clases seleccionadas como entidades independientes, justificando su elección en base a los requerimientos funcionales y no funcionales extraídos del enunciado del proyecto "Desafío II: Mercado de estadías hogareñas UdeAStay". El diseño busca modelar eficientemente las entidades principales y sus interacciones, facilitando la gestión de alojamientos, reservaciones, anfitriones y huéspedes.

Index Terms—Programación Orientada a Objetos, Diseño de Clases, UML, UdeAStay, C++, Gestión de Reservas, Ingeniería de Software.

I. INTRODUCCIÓN

El proyecto UdeAStay tiene como objetivo desarrollar un sistema para la administración de un mercado de estadías hogareñas, utilizando los principios de la Programación Orientada a Objetos (POO), con la particularidad de evitar el uso de mecanismos de herencia para las entidades principales de roles de usuario. El sistema debe permitir la gestión eficiente de alojamientos, reservaciones, los anfitriones y los huéspedes. Este documento se enfoca en la fase de análisis y diseño, específicamente en la definición y justificación de las clases que conforman la arquitectura de la solución, basándose en el documento "Desafío II: Mercado de estadías hogareñas UdeAStay" (referido en adelante como *.el enunciado*).

El modelado correcto de las entidades del dominio del problema es fundamental para construir un software robusto y mantenible. La elección de clases, sus atributos, métodos y relaciones es el pilar de una solución POO efectiva, buscando la abstracción y encapsulación adecuadas para representar el sistema.

II. ANÁLISIS DEL PROBLEMA

El enunciado describe un sistema para UdeAStay, una compañía que aspira a posicionarse en la promoción de la actividad turística y la regulación del comercio de alojamientos en Antioquia. Los requerimientos principales identificados para el diseño de clases son:

- Gestión de información detallada de **alojamientos**: nombre, código, anfitrión responsable, ubicación, tipo, dirección, precio, amenidades, y fechas reservadas.

- Gestión de información de **anfitriones**: número de documento, antigüedad, puntuación y alojamientos que administra.
- Gestión de información de **huéspedes**: número de documento, antigüedad, puntuación e información de sus reservas.
- Gestión de **reservaciones**: fecha de entrada, duración, código, detalles del alojamiento y huésped, pago, monto, y anotaciones.
- Implementación de **funcionalidades esenciales** como carga/actualización de datos, ingreso a la plataforma por perfil, reserva de alojamiento, anulación de reservaciones, consulta de reservaciones por anfitriones y actualización del histórico.
- Medición automática del consumo de recursos.

Estos requerimientos implican la identificación de entidades distintas con responsabilidades claras. La restricción de no usar herencia significa que los roles de usuario (Anfitrión y Huésped) se modelarán como clases separadas y completas.

III. JUSTIFICACIÓN DEL DISEÑO DE CLASES

Considerando la restricción de no utilizar herencia, se propone la siguiente estructura de clases:

III-A. Clase *Anfitrión*

Justificación: El anfitrión es uno de los dos roles principales de usuario en el sistema, con responsabilidades y datos específicos para la oferta y gestión de alojamientos. Al no utilizar una clase base común, *Anfitrión* encapsulará toda su información y comportamiento de manera independiente.

■ Atributos Principales:

- 'numeroDocumento' (String): Identificador único del anfitrión.
- 'credenciales' (Object): Para el proceso de inicio de sesión (e.g., nombre de usuario, contraseña).
- 'antigüedadPlataformaMeses' (int): Tiempo del anfitrión en la plataforma.
- 'puntuacion' (float): Calificación del anfitrión (0-5.0).
- 'alojamientosAdministrados' (Lista de 'Alojamientos'): Referencias a los alojamientos que gestiona.

■ Métodos Principales:

- ‘iniciarSesion(documento, password)’ : Verifica las credenciales para el acceso.
- ‘obtenerInformacionPersonal()’ : Devuelve los datos del perfil del anfitrión.
- ‘agregarAlojamiento(nuevoAlojamiento: Alojamiento)’ : Añade un nuevo alojamiento a su lista.
- ‘eliminarAlojamiento(codigoAlojamiento: String)’ : Elimina un alojamiento.
- ‘consultarReservacionesActivas(fechaInicio: Date, fechaFin: Date)’ : Muestra las reservaciones activas para sus alojamientos.
- ‘anularReservacion(codigoReservacion: String)’ : Permite al anfitrión anular una reservación asociada a sus alojamientos.

- **Modelado Directo de Rol:** Representa directamente la entidad Anfitrión como se describe en el problema, con todas sus características necesarias para interactuar con el sistema en su rol específico de proveedor de alojamientos.

III-B. Clase Huesped

Justificación: El huésped es el otro rol principal de usuario, enfocado en la búsqueda y reserva de alojamientos. Como clase independiente, Huesped contendrá toda la información y lógica necesaria para sus interacciones.

- **Atributos Principales:**
 - ‘numeroDocumento’ (String): Identificador único del huésped.
 - ‘credenciales’ (Object): Para el proceso de inicio de sesión.
 - ‘antiguedadPlataformaMeses’ (int): Tiempo del huésped en la plataforma.
 - ‘puntuacion’ (float): Calificación del huésped (0-5.0).
 - ‘reservasRealizadas’ (Lista de ‘Reserva’): Historial de reservaciones del huésped.
- **Métodos Principales:**
 - ‘iniciarSesion(documento, password)’ : Verifica las credenciales para el acceso.
 - ‘obtenerInformacionPersonal()’ : Devuelve los datos del perfil del huésped.
 - ‘buscarAlojamientosDisponibles(criterios: Object)’ : Busca y filtra alojamientos.
 - ‘buscarAlojamientoPorCodigo(codigoAlojamiento: String)’ : Busca un alojamiento específico.
 - ‘realizarReserva(alojamientoSeleccionado: Alojamiento, ...)’ : Crea una nueva reservación.
 - ‘anularReservacion(codigoReservacion: String)’ : Permite al huésped anular una de sus reservaciones.
 - ‘consultarMisReservaciones()’ : Muestra las reservaciones del huésped.
- **Modelado Directo de Rol:** Corresponde a la entidad “Huésped” como se describe en el problema, con la capacidad de realizar búsquedas, reservaciones y gestionar su información de manera autónoma dentro del sistema. La restricción de no solapamiento de fechas en sus reservas es una lógica de negocio importante asociada a esta clase.

III-C. Clase Alojamiento

Justificación: Esta es una de las entidades centrales del sistema UdeAStay, representando las propiedades que se ofrecen en alquiler. Su diseño permanece fundamentalmente igual, ya que no dependía de la herencia de roles de usuario.

- **Atributos Principales:**
 - ‘codigoIdentificador’ (String): Código único del alojamiento.
 - ‘nombre’ (String): Nombre descriptivo.
 - ‘anfitriónResponsable’ (Referencia a ‘Anfitrión’): El anfitrión dueño.
 - ‘departamento’, ‘municipio’, ‘tipo’, ‘direccion’ (String).
 - ‘precioPorNoche’ (float).
 - ‘amenidades’ (Lista de Strings).
 - ‘fechasReservadas’ (Lista de ‘RangoFechas’ o similar).
- **Métodos Principales:**
 - ‘estaDisponible(fechaInicio: Date, numeroNoches: int)’ : Verifica disponibilidad.
 - ‘agregarReserva(reserva: Reserva)’ : Añade una reserva a sus fechas.
 - ‘eliminarReserva(codigoReserva: String)’ : Quita una reserva.
 - ‘obtenerInformacionDetallada()’ : Devuelve detalles del alojamiento.
- **Entidad Fundamental:** Sigue siendo crucial para el sistema, encapsulando toda la información y el estado de una propiedad.

III-D. Clase Reserva

Justificación: Modela la acción y el registro de una reservación de un alojamiento por parte de un huésped. Su diseño tampoco se ve afectado directamente por la eliminación de la herencia entre usuarios.

- **Atributos Principales:**
 - ‘codigoReserva’ (String): Identificador único, gestionado automáticamente.
 - ‘fechaEntrada’ (Date), ‘duracionNoches’ (int).
 - ‘alojamientoReservado’ (Referencia a ‘Alojamiento’).
 - ‘huesped’ (Referencia a ‘Huesped’).
 - ‘metodoPago’, ‘fechaPago’, ‘montoTotal’.
 - ‘anotacionesHuesped’ (String).
- **Métodos Principales:**
 - ‘calcularFechaSalida()’ : Determina la fecha de finalización.
 - ‘confirmarReserva()’ : Marca la reserva como confirmada.
 - ‘obtenerComprobante()’ : Genera detalles de la confirmación.
- **Entidad Transaccional Clave:** Sigue siendo el nexo entre un huésped y un alojamiento para un período específico, con todas las validaciones y datos asociados.

III-E. Clase *SistemaUdeAStay* (o *GestorPrincipal*)

Justificación: Se necesita una clase orquestadora que maneje las interacciones generales del sistema, la lógica de negocio principal y la interfaz con el usuario (menús).

■ Atributos Principales:

- 'listaAlojamientos' (Lista de 'Alojamiento').
- 'listaAnfitriones' (Lista de 'Anfitrión').
- 'listaHuespedes' (Lista de 'Huesped').
- 'historicoReservas' (Lista de 'Reserva').

■ Métodos Principales:

- 'cargarDatosDesdeArchivos()' / 'actualizarDatosEnArchivos()'.
- 'autenticarUsuario(documento: String, password: String, rolIndicado: String)': Verifica credenciales contra la lista de Anfitriones o Huespedes.
- 'gestionarReservaAlojamiento(huesped: Huesped, ...)'
- 'actualizarArchivoHistorico(fechaCorte: Date)'.
- 'medirConsumoRecursosInicio()' / 'medirConsumoRecursosFin()'.
- 'mostrarMenuPrincipal()' y menús específicos por rol.

- **Control Centralizado:** Su rol se vuelve aún más importante para gestionar las colecciones separadas de anfitriones y huéspedes y coordinar las operaciones entre todas las entidades.

entre Anfitrión y Huesped (como 'numeroDocumento', 'antiguedadPlataformaMeses', 'puntuacion'), cada clase encapsula completamente las responsabilidades y datos de su respectivo rol. Esta estructura busca proporcionar una base sólida y cohesiva para la implementación de la solución en C++. La correcta definición de constructores, destructores, métodos de acceso y la sobrecarga necesaria, junto con una gestión eficiente de la memoria y las estructuras de datos (sin uso de STL y con memoria dinámica), serán cruciales para cumplir con todos los objetivos del desafío.

IV. RELACIONES ENTRE CLASES

Con la eliminación de la herencia para los roles de usuario, las relaciones se reconfiguran de la siguiente manera:

■ Asociación y Agregación:

- Un Anfitrión administra uno o muchos Alojamientos ("1 a N"). Un Alojamiento tiene un Anfitrión responsable ("1 a 1").
- Un Huesped realiza cero o muchas Reservas ("1 a N"). Una Reserva pertenece a un Huesped ("1 a 1").
- Un Alojamiento puede tener múltiples Reservas a lo largo del tiempo ("1 a N"). Una Reserva es para un Alojamiento específico ("1 a 1").

Estas relaciones modelan cómo las instancias de las clases se conectan e interactúan.

- La clase *SistemaUdeAStay* agrega y gestiona colecciones separadas de Anfitrión, Huesped, Alojamiento, y Reserva.

Estas relaciones son esenciales para mantener la integridad de los datos y para la correcta ejecución de las funcionalidades requeridas por el sistema.

V. CONCLUSIÓN

Este enfoque se deriva directamente del análisis de los requisitos funcionales y las entidades descritas en el enunciado del proyecto UdeAStay, adaptado a las restricciones de diseño. Aunque se pueda presentar duplicación de atributos comunes